Verifier-Grounded Generate-Verify-Repair for Intent→Configuration NetOps Tasks: A Paired Mininet Emulation Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a provenance-traced paired confirmatory study (N = 200 intents; 400 paired episodes) to evaluate whether a deterministic verifier-grounded generate–verify–repair (GVR) loop reduces operationally detectable policy-violation errors when a hosted LLM produces device configuration sequences from operator intent in a Mininet emulation. Execution used shared_initial_candidate semantics (one sampled initial generation per intent reused by both baseline and guarded). Baseline success (no detected policy violation) = 196/200 (98%); guarded success = 200/200 (100%). Paired risk difference (guarded - baseline) = 0.02; preregistered exact McNemar two-sided p = 0.125 (primary confirmatory test). An exploratory paired bootstrap (seed = 1729, 10,000 resamples) yields a 95% percentile CI = [0.005, 0.04]. Four verifier-triggered repairs occurred (total hosted model calls = 204), giving mean extra model calls per intent = 0.02 and mean extra calls per avoided violation = 1.0. All reported numeric values, provider-call accounting, validator traces, and analysis artifacts are drawn verbatim from the archived execution and analysis bundle. We interpret the preregistered confirmatory result conservatively and discuss construct, internal, and external validity limits and operational implications.**

I. Introduction

Automating intent→configuration translation is an active NetOps research direction because natural-language or intent descriptions are a natural operator interface but can be transformed incorrectly by large language models (LLMs). Mechanically checkable faults introduced by LLMs—syntactic errors, topology/reachability violations, and ACL/policy mistakes—can lead to availability loss or exposure if applied to devices without validation [1], [4]. Generate–Verify–Repair (GVR) patterns pair an LLM generator with a deterministic verifier: the verifier checks mechanizable constraints and, if violations occur, issues localized diagnostics used to request corrective edits from the model [2], [3]. Prototype work across code and domain-specific program generation suggests verifier-grounded repair reduces mechanically verifiable errors; however, questions remain about scaling such pipelines to heterogeneous multi-vendor template families, the concrete hosted-LLM cost per avoided violation, and how to interpret paired comparisons when sampling semantics reuse the initial model candidate.

Research question and contribution. Our preregistered research question is: Does a verifier-grounded generate–verify-repair loop significantly reduce operationally detectable policy-violation errors produced by a hosted LLM when generating network configurations for operator intents across heterogeneous multi-vendor template families and topology patterns in a Mininet-based emulation? Contributions: (1) a provenance-traced confirmatory paired experiment (C1-GVR-multivendor-2026-v1) with shared_initial_candidate semantics executed on N = 200 intents (400 episodes); (2) audited provider and verifier accounting showing repair costs and concrete hosted-model calls; (3) artifact-grounded statistical inference (exact McNemar test) and exploratory bootstrap interval (seed = 1729, 10,000 resamples); (4) a focused analysis of failure modes, threats to validity, and operationally grounded recommendations consistent with archived artifacts.

II. Related Work

Recent surveys and reviews document rapid growth in LLM-for-NetOps and AIOps research while emphasizing recurring evaluation gaps—benchmark provenance, verifier integration, and limited multi-vendor emulation evidence [5], [6]. Empirical prototypes and benchmarks show that LLMs can draft device configurations or configuration-style artifacts, but they frequently produce mechanically checkable faults (syntax, missing or misordered commands, and reachability or policy violations) when evaluated on configuration translation tasks; these failure modes have been observed in domain-specific studies and NetOps benchmarks [1], [4].

A separate strand of work proposes and demonstrates verifier-grounded closed-loop architectures (generate→verify→repair) in coding and domain-specific program generation settings [2], [3]. These proposals and single-case or small-scale empirical demonstrations indicate that when (i) a deterministic verifier can express the relevant mechanizable constraints, and (ii) the verifier provides localized, actionable diagnostics, a bounded repair policy can reduce the incidence of mechanically verifiable errors. However, the cited records are prototype-scale: effectiveness depends on verifier expressivity, the class of detectable violations, and close coupling between verifier diagnostics and repair prompts; they do not by themselves establish broad production-scale safety or generality across heterogeneous vendor templates and topologies [2], [3].

Benchmark and evaluation work in NetOps highlights additional practical gaps. NetConfEval and related evaluations demonstrate feasibility of measuring LLM capability on configuration tasks, but these efforts commonly lack provenance-traced verifier pipelines and do not reliably quantify multi-vendor template heterogeneity or cost metrics tied to verifier-triggered repairs [4]. The surveys cited above call for richer provenance, clearer operationally meaningful metrics, and higher-fidelity emulation in order to make comparative claims about reliability and to expose residual semantic failures that deterministic verifiers may miss [5], [6].

Taken together, the verified literature supports three tempered conclusions that motivate this study: (1) LLMs can produce useful draft configurations but commonly make mechanically checkable errors; (2) verifier-grounded repair is a recurring mitigation pattern with promising single-case evidence but limited large-scale cross-vendor validation; and (3) more provenance-traced, operationally framed experiments are required to quantify repair cost tradeoffs and the residual semantic errors outside verifier coverage. Our preregistered paired design directly targets these gaps by providing provenance-traced accounting, an explicit shared_initial_candidate execution semantics to isolate verifier/repair effects, and per-intent accounting of repair calls and final validator outcomes.

III. Methodology

Preregistration and estimand. The experiment was preregistered as C1-GVR-multivendor-2026-v1 before any model calls (preregistration SHA-256: 9e0fc992665835975b01289f7841926ef6abf358 4252c5d0459ba0ecf166bb50). Primary estimand: paired_success_rate_difference_guarded_minus_baseline (probability that an intent yields zero operationally detectable violations under guarded minus baseline), tested with an exact McNemar two-sided test at $\alpha = 0.05$. Predeclared exploratory estimators include a paired bootstrap percentile CI (bootstrap_seed = 1729; resamples = 10,000) and descriptive cost metrics (mean extra hosted model calls per intent and mean extra calls per avoided violation).

Sampling, strata, and shared_initial_candidate semantics. Task selection was deterministic and stratified across three synthetic vendor-template families and four topology patterns (12 strata) to exercise template heterogeneity. N = 200 unique operator intents were fixed prior to model calls. Execution used shared_initial_candidate semantics: exactly one initial sampled generation per intent was produced and reused by both baseline and guarded episodes. Under these semantics baseline outcome is the deterministic validator verdict on that shared initial candidate; the guarded outcome equals the final guarded pipeline result after deterministic validation and any permitted repair. The preregistration explicitly declared that independent constrained-prompt arms were not executed; therefore paired differences can arise only from deterministic validator or repair steps, not from different first-pass samples or prompt templates.

Model configuration, sampling notation, and budget. Declared model (as archived) is gpt-5-mini (provider recorded as openai_responses). The archived model_configuration.json records temperature = null/unset; we explicitly state that null/unset is not evidence of deterministic provider sampling and does not imply temperature = 0. Planned budget enforced in the adapter: planned_episode_count = 400, initial_generation_calls_per_task = 1, maximum_repair_calls_per_task = 1, maximum_model_calls = 400.

Deterministic verifier and repair policy. deterministic_netops_validator_v5 is a rule-based verifier combining (i) syntactic parsing and command pattern checks, (ii) required-command sequencing and workflow policy checks (per-task payload), (iii) reachability/topology checks against the emulated Mininet state, and (iv) encoded ACL/policy checks derived from the task payload. The scorer rule full_intent_constraint_validation yields a binary success = "no detected violations" if the final configuration satisfies all mechanizable constraints. Guarded pipeline behavior: if the validator flags violations, a single verifier-triggered repair call may be issued; repair prompts contain the verifier's localized diagnostics requesting corrective edits. Repair calls were permitted only when the validator reported violations, and up to one repair call per intent was allowed per the preregistration.

Scoring, missingness, and analysis plan. Primary analysis: exact McNemar two-sided test on paired binary success indicators. Missingness and exclusions were predeclared: duplicate tasks (detected via deterministic hashing) or unrecoverable infra failures would be excluded and reported; ambiguous validator outputs were conservatively treated as violations in the primary analysis. Exploratory analyses included the paired bootstrap percentile CI (seed = 1729, 10,000 resamples) and descriptive cost accounting. Analysis code and seeds are archived.

IV. Results

Execution completion and deterministic accounting. The preregistered run completed without infrastructure exclusions: planned_episode_count = 400, completed_episode_count = 400, complete_pair_count = 200, failed_episode_count = 0 (execution manifest). Audited provider accounting (deterministic_reconciliation) reports hosted_model_call_event_count = 204; this decomposes deterministically into stage_counts = {shared_initial_generation: 200, repair: 4} and condition_counts recorded as {shared: 200, guarded: 4} in the manifest. Cache statistics show cache_hit_count = 0 and cache_miss_count = 204, confirming that every required hosted call executed and that the four repair calls were distinct provider calls. All deterministic consistency checks passed in the reconciliation artifacts.

Paired binary outcomes and primary confirmatory test. The preregistered paired contingency (analysis/paired_contingency_table.csv) is n11 = 196, n10 = 4, n01 = 0, n00 = 0. These cells yield baseline_success_rate = 196/200 = 0.98 and guarded_success_rate = 200/200 = 1.00; the paired risk difference (guarded - baseline) = 0.02. The preregistered exact McNemar two-sided test on these paired indicators returned p = 0.125 (analysis results). Under the preregistered inferential contract this p-value means we do not reject the null at α = 0.05.

Repair events, candidate reuse, and per-intent cost accounting. The stage decomposition (200 shared initial generation events + 4 repair events) matches the model-call audit and demonstrates that the guarded pipeline invoked repairs for exactly four intents. From these audited counts we compute total_model_calls = 204, mean_model_calls_per_intent = 204/200 = 1.02, mean_extra_calls_per_intent_due_to_repairs = 4/200 = 0.02. Because each repair converted one failing baseline into a passing guarded outcome in the archived contingency, the observed mean_extra_calls_per_avoided_violation = 4 repairs / 4 avoided violations = 1.0. These arithmetic derivations follow directly from the archived execution manifest and deterministic reconciliation artifacts.

Shared_initial_candidate evidence and representative artifact substantiation. Representative task artifacts included in the evidence bundle show explicit reuse of the same sampled initial generation by both baseline and guarded episodes. For example, the representative tasks (task-000004, task-000005, task-000006, etc.) exhibit identical shared_initial_candidate contents and identical shared_initial response file SHA-256 values across their baseline and guarded response artifacts. The archived validator_trace fields for the discordant pairs record validation_before entries showing violations and validation_after entries showing repair_applied = true with final validation_after.valid = true; the bundle therefore substantiates that the four discordant cases were resolved by verifier-triggered repairs operating on the identical initial candidate.

Diagnostic pattern observed in discordant pairs. In every archived discordant pair the validator_trace shows a mechanizable violation detected in validation_before (violation_count > 0 and violation_codes returned by deterministic_netops_validator_v5) and a subsequent repair application that produced a final configuration satisfying full_intent_constraint_validation. The manifest and per-task scoring artifacts indicate that repairs were only issued for intents where the verifier produced explicit, localized diagnostics; no repair calls occurred for intents without detected violations. The archived traces therefore support the causal pathway: identical initial candidate $\rightarrow$ verifier detects mechanizable violation $\rightarrow$ single permitted repair call $\rightarrow$ final guarded configuration passes deterministic validation.

Exploratory bootstrap interval and interpretation caution. The archived exploratory paired bootstrap (bootstrap_seed = 1729; resamples = 10,000) produced a 95% percentile bootstrap CI for the paired risk difference = [0.005, 0.04]. We report this interval as exploratory only. Practically, the bootstrap percentile CI in paired binary settings relies on resampling paired outcomes and can be unstable when the total number of discordant pairs (n10 + n01) is small; here n10 + n01 = 4 is small, which reduces the bootstrap's reliability as a standalone uncertainty summary. Accordingly, we treat the exact McNemar test as the prespecified confirmatory inference (p = 0.125) and use the bootstrap interval only to convey plausible effect-size magnitudes suggested by the archived data.

Power context and observed baseline prevalence. The preregistered power calculation targeted a 10 percentage-point absolute reduction with N $\approx$ 157 (McNemar approximation) and noted that with N = 200 the detectable absolute risk difference under the same assumptions would be $\approx$8 percentage points. The observed baseline failure prevalence (2%) was substantially lower than scenarios that motivated the target effect size, which reduced the study's power to detect small absolute improvements and produced the small discordant count (4) observed. This lower than anticipated baseline error rate is an empirical fact recorded in the archived condition summary and underlies the conservative interpretation of the null result from the preregistered test.

Audit reconciliation and reproducibility pointers. All numerical claims above (paired cell counts, model_call totals, decomposition into shared initial and repair stages, bootstrap parameters, and seeds) are directly traceable to archived artifacts: execution/raw_results.jsonl, analysis/paired_contingency_table.csv, deterministic_reconciliation.json, execution/model_configuration.json, and analysis/analysis_log.jsonl. The execution manifest records started_at_utc and completed_at_utc timestamps for the run and the preregistration SHA-256; the analysis artifacts record the exact bootstrap_seed = 1729 and bootstrap_resamples = 10000 used to produce the exploratory interval. Reproducibility of the primary contingency counts does not require re-sampling because shared_initial_candidate semantics ensured exactly one initial sampled generation per intent was reused by both conditions; the archived response files and validator traces substantiate this reuse.

Summary of result-section evidence. In short: (1) the formal preregistered confirmatory test did not reject the null hypothesis at α = 0.05 (exact McNemar p = 0.125); (2) deterministic verifier-triggered repairs converted four failing shared initial candidates into passing final configurations, as shown by per-task validator_trace entries; (3) the total hosted model-call overhead for these repairs was low (4 additional calls across 200 intents, mean extra calls per intent = 0.02, mean extra calls per avoided violation = 1.0); (4) the exploratory bootstrap CI [0.005, 0.04] suggests small plausible effect magnitudes but must be interpreted cautiously given the sparse discordant pair count. These findings are reported verbatim from the archived execution and analysis bundle and are discussed further in the Discussion section.

V. Discussion

Causal interpretation under shared_initial_candidate semantics. Because execution used shared_initial_candidate semantics (one sampled initial generation per intent reused by baseline and guarded), paired improvements arise solely from deterministic validator/repair actions. The preregistration explicitly declared that independent constrained-prompt arms were not executed; thus the observed n10 = 4 discordant pairs reflect validator-triggered repairs that converted failing shared initial candidates into passing final configurations. Any statements attributing improvements to prompt-engineering or differing first-pass samples would be incorrect for this run.

Construct validity and verifier coverage. The deterministic_netops_validator_v5 covers syntactic correctness, required command sequencing, reachability/topology checks against the emulated state, and encoded ACL/policy checks derived from per-task payload constraints. This verifier can only detect violations expressible in its rules; higher-order semantic misinterpretations not captured by those encodings remain undetectable and are identified in our limitations. Increasing verifier expressivity (formal policy ontologies, broader semantic checks) is a direct next step to reduce residual false negatives.

Internal validity and nondeterminism. The archived model_configuration.json records temperature = null/unset; we reiterate that null/unset does not imply temperature = 0 or deterministic provider sampling. The execution manifest's provider audit (hosted_model_call_event_count = 204, stage_counts, and condition_counts) was reconciled deterministically (all deterministic consistency checks passed) and confirms that cross-condition cache reuse did not occur; this audit supports the claim that baseline outcomes are the validator verdict on the identical shared initial generation used by guarded episodes.

Operational implications (bounded). Within the constrained Mininet emulation, synthetic templates, and validator rule set, a small repair budget (four repairs) removed the mechanically detectable baseline violations at very low hosted-model cost. This artifact-grounded observation suggests that when a deterministic verifier exists and provides localized actionable diagnostics, a bounded repair policy can remove template-driven mechanically checkable faults in similar emulated settings. We do not claim production readiness, multi-model generality, nor coverage for semantic errors outside the verifier.

Threats to validity and recommendations. Internal validity—shared_initial_candidate semantics were predeclared and executed; this prevents attributing gains to independent re-sampling. Construct validity—the verifier cannot detect all semantic errors; extend verifier coverage and measure false negatives. External validity—single declared model (gpt-5-mini) and synthetic templates in Mininet limit generalization; evaluate additional hosted models and hardware testbeds. Conclusion validity—small discordant counts reduce power; future preregistered work could target higher baseline failure prevalence scenarios or increase N. Based on archived evidence we recommend: (1) expanding deterministic verifier expressivity to formalize more semantic/policy ontologies; (2) performing paired designs that also execute independent multi-sample initial generations to separate prompt/sampling effects from repair effects; and (3) repeating provenance-traced evaluations across multiple hosted models and template families.

VI. Limitations

- Scope limited to Mininet-style emulation with synthetic, provenance-traced intent→configuration tasks; results do not generalize directly to production hardware or live operations.
- Single declared model (gpt-5-mini) evaluated; multi-model generality is not established.
- Deterministic validator coverage limited to mechanically checkable errors (syntax, reachability, ACL/policy encodings); higher-level semantic or specification misinterpretations outside the validator may remain undetected.
- Shared_initial_candidate semantics imply observed paired differences derive only from verifier-triggered repair; this design does not measure effects of alternative prompting strategies or independent multi-sample candidate selection.
- Observed confirmatory effect (2 percentage points) did not reach statistical significance at α = 0.05 for N = 200 (exact McNemar p = 0.125); low baseline failure prevalence (2%) reduced power to detect small absolute improvements under some preregistered scenarios.

VII. Conclusion

We preregistered and executed a provenance-traced paired confirmatory study (C1-GVR-multivendor-2026-v1) using shared_initial_candidate semantics to measure whether a deterministic verifier plus bounded repair reduces mechanically detectable policy violations for intent→configuration tasks in a Mininet emulation. The preregistered exact McNemar two-sided test did not reject at α = 0.05 (p = 0.125). Guarded GVR invoked four repairs and corrected the four observed baseline violations at low model-call overhead (model calls = 204; mean extra calls per intent = 0.02), yielding a paired risk difference estimate of 0.02 and an exploratory paired bootstrap CI = [0.005, 0.04] (bootstrap_seed = 1729, resamples = 10,000). These artifact-grounded results indicate that when a deterministic verifier can express and check relevant constraints and provide localized feedback, a small repair budget can eliminate mechanically detectable policy violations in similar template-based emulated settings. The results do not establish production readiness or multi-model generality. All reported numbers, provider accounting, validator traces, and analysis artifacts are drawn verbatim from the archived execution and analysis bundles referenced in the preregistration and execution manifests.

References [1] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", Proceedings of the ACM, 2023, doi: 10.1145/3626111.3628194. [2] R. Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", SSRN, 2026, doi: 10.2139/ssrn.6754899. [3] Z. Ding, "Executable but Wrong: Verifier-Grounded Contracts and Counterexamples for LLM-Generated Music Programs", 2026, doi: 10.31224/7995. [4] C. Wang, M. Scazzariello, A.

Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", Proceedings of the ACM, 2024, doi: 10.1145/3656296. [5] S. Y. Long et al., "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models", IEEE Communications Surveys & Tutorials, 2025, doi: 10.1109/comst.2025.3526606. [6] L. Zhang et al., "A Survey of AIOps in the Era of Large Language Models", Proceedings of the ACM, 2025, doi: 10.1145/3746635.

Disclosure Statement

Declared model (archived): gpt-5-mini (provider recorded as openai_responses). Adapter family: hosted_netops_gvr_v1. Deterministic validator: deterministic_netops_validator_v5. Task generator: deterministic_netops_task_generator_v5. Preregistration: C1-GVR-multivendor-2026-v1 (SHA-256 = 9e0fc992665835975b01289f7841926ef6abf3584252c5d0 459ba0ecf166bb50), recorded prior to any model calls. Initial master prompt archived at provenance/master_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196 add77f5acdf1582bb39b15ba). Human role: a human Research Director selected the topic family and constrained the initial master prompt prior to the autonomy lock; after the lock the AI autonomously performed literature verification, preregistration, experiment execution, analysis, peer review, revision, and manuscript drafting; no human manually edited technical manuscript content after the autonomy lock. Sampling semantics: execution used shared_initial_candidate (exactly one sampled initial generation per intent was produced and reused by both baseline and guarded); archived model configuration records temperature = null/unset (null/unset is not evidence of deterministic provider sampling and does not imply temperature = 0). Provider accounting (audited): hosted_model_call_event_count = 204 decomposed as 200 shared initial generations + 4 repair calls. Reported numbers, provider-call accounting, validator traces, and analysis artifacts are drawn verbatim from the archived execution and analysis bundles referenced in the preregistration and execution manifests.

References

[1] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194

[2] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899

[3] Zhengyang Ding, "Executable but Wrong: Verifier-Grounded Contracts and Counterexamples for LLM-Generated Music Programs", 2026, 10.31224/7995

[4] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296

[5] S.Y. Long, Jingjing Tan, Bomin Mao, Fengxiao Tang, Yangfan Li, Ming Zhao, Nei Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models", 2025, 10.1109/comst.2025.3526606

[6] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635